

SparkUI-Parser: 通过鲁棒定位与解析增强 GUI 感知 Hongyi Jing^{2 *}, Jiafu Chen^{1 *}, Chen Rao¹, Ziqiang Dang², Jiajie Teng², Tianyi Chu¹, JunCheng Mo¹, Shuo Fang², Huaizhong Lin¹, Rui Lv^{2 †}, Chenguang Ma^{2 †}, Lei Zhao^{1 † 1}

浙江大学 2 蚂蚁集团 {jinghongyi.jhy, dijun.lr, chenguang.mcg}@antgroup.com, {chenjiafu, cszhl}@zju.edu.cn
arXiv:2509.04908v1 [cs.AI] 5 Sep 2025

摘要 现有用于 GUI 感知的多模态大语言模型 (MLLMs) 已经取得了很大进展。然而, 先前方法中仍然存在以下挑战: 1) 它们基于文本自回归机制对离散坐标进行建模, 这导致定位准确率较低且推理速度较慢。2) 它们只能定位预定义的元素集合, 无法解析整个界面, 这阻碍了其广泛应用以及对下游任务的支持。为解决上述问题, 我们提出 SparkUI-Parser, 这是一种新颖的端到端框架, 可同时实现更高的定位精度和对整个界面的细粒度解析能力。具体而言, 我们并非采用基于概率的离散建模, 而是在预训练的多模态大语言模型

(MLLM) 基础上, 结合额外的 token 路由器和坐标解码器, 对坐标进行连续建模。这有效缓解了 MLLMs 的离散输出特性和逐 token 生成过程固有的局限性, 从而同时提升准确率和推理速度。为进一步增强鲁棒性, 我们引入了一种基于改进匈牙利匹配算法的拒绝机制, 使模型能够识别并拒绝不存在的元素, 从而减少误报。此外, 我们提出 ScreenParse, 这是一个经过严格构建的基准, 用于系统评估 GUI 模型在多样化场景中的结构化感知能力。大量实验表明, 我们的方法在 ScreenSpot、ScreenSpot-v2、CAGUI-Grounding 和 ScreenParse 基准上始终优于 SOTA 方法。相关资源可在 <https://github.com/antgroup/SparkUI-Parser> 获取。

1

引言

面向 GUI 的 MLLMs 能够整合视觉和文本数据, 以理解图形用户界面 (GUI) 并与之交互, 为构建 GUI 智能体提供了坚实基础。GUI 智能体有潜力自主操作广泛的设备, 从而将人机交互从完全人工的过程转变为自动化和委托式 workflow。由于通用 MLLMs 的大多数训练* 这些作者贡献相同。†

通讯作者。Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

数据是自然图像, 它们在 GUI 图像上的感知能力不足, 这限制了它们在 GUI 特定语境中的有效性。自然图像通常包含具有多样对象和背景的复杂场景, 而 GUI 图像则更具结构性, 包含文本、按钮和输入框等具有特定功能和布局的元素。对于 GUI 场景感知和高级任务, 理想的 GUI MLLMs 不仅需要理解各种设备上不断变化且信息密度高的界面, 还需要准确执行基本操作, 例如理解界面元素的语义并输出精确坐标。为构建这种强大的 GUI MLLMs, 已有若干工作

(Cheng et al. 2024; Xu et al. 2024; Gou et al. 2024) 在 GUI 界面和任务上对 MLLMs 进行了后训练。它们专注于通过定位 GUI 界

面上的图标或文本来提升 GUI 感知, 并已展现出有前景的结果。然而, 在感知 GUI 界面方面, 这些先前方法仍然存在一些局限。首先, 它们直接在通用 MLLMs 上进行微调, 其中基于离散 token 概率分布的文本自回归机制会导致结果不准确且效率较低。因此, 在定位任务中, 它可能无法达到传统模型的准确率。此外, 当需要快速响应时间时, 高精度坐标的逐 token 预测方式可能成为瓶颈。其次, 它们只关注定位指定元素并与之交互, 无法对整个用户界面上更广泛的上下文以及元素之间的关系提供详细的解析式感知。第三, 当需要定位不存在的元素时, 先前模型会返回错误位置或生成无关响应, 从而导致下游任务失败, 这会显著影响用户体验和系统可靠性。为解决上述局限, 我们提出一种新颖的端到端模型, 以在一个模型内实现鲁棒定位和解析。具体而言, 我们设计了一个“先路由后预测”的框架, 用于高效处理视觉和语言信息, 该框架由一个 MLLM、一个 token 路由器、一个视觉适配器、一个坐标解码器以及一个仅用于训练阶段的额外元素匹配器组成。来自 MLLM 的输出 token 由 token 路由器分类为文本 token 和视觉定位 token。文本 token 被解码为元素语义, 而视觉定位 token 结合

box2

box3

box1

请提供该用户界面中以下指令的位置: 转账、扫描二维码、播放音乐。
扫描二维码 box2 播放音乐 box3

请解析该用户界面中的所有元素。Qwen2.5-VL

Qwen2.5-VL

目标缺失, 并对不存在的目标提供无关输出。转账 box1 扫描二维码 box2 播放音乐在界面上不存在

我们的方法

正确处理指令中的所有元素。

(a) 多目标定位与不存在元素的拒绝定位

我们的方法

添加新地点 icon1 AC010ML的家 text1 6 台设备 text2 ... 场景 icon10 场景 text19 我 icon11 我 text20

添加新地点 icon1 AC010ML的家 text1 选择地点 icon2 6 台设备

text2 ... 场景 icon14 场景 text21 我 icon15 我 text22

元素缺失、冗余元素以及定位偏差。

(b) 截图解析

准确解析所有元素。

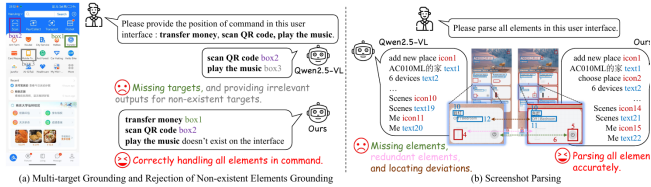


Figure 1: A demonstration of multi-target grounding, rejection of non-existent elements grounding, and parsing on Qwen2.5-VL and Ours.

图 1：在 Qwen2.5-VL 和我们的方法上进行多目标定位、拒绝不存在元素定位以及解析的演示。来自视觉适配器的视觉特征由坐标解码器处理以进行定位。轻量级坐标解码器弥合了多模态大语言模型（MLLM）基于概率的离散生成与图像空间连续特性之间的差距，并提供了更高精度的定位能力和更高效的推理。元素匹配器用于确保在用户界面的多目标定位与解析中，元素语义和坐标被正确匹配。借助自建的解析数据集，我们训练模型进行解析，以提取所有元素的全面表示，包括其语义及对应位置，从而赋予模型对整个用户界面的解析能力。此外，我们还引入拒绝 token 来表示用户界面中不存在的元素。当拒绝 token 出现时，其坐标解码过程会被直接跳过，从而避免不必要的计算，并降低生成错误坐标的风险。鲁棒定位与界面解析的演示如图 1 所示。总而言之，我们的主要贡献如下：

- 据我们所知，我们是首个为 GUI 感知引入端到端 MLLM 的工作，该模型能够同时也在用户界面上实现鲁棒定位与解析，提供对语义和结构的全面感知。
- 我们提出了一种新的“先路由再预测”框架，将离散词表坐标建模转换为连续空间坐标值，以实现更好的 GUI 定位与解析。通过分别处理元素的语义和坐标，我们的方法平均将定位精度提高约 3%，并使定位和解析速度分别平均提升 5 倍和 4 倍。
- 我们提出了一个用于 GUI 解析的基准 ScreenParse，它用于评估模型在定位特定元素以及感知用户界面整体结构方面的性能。此外，我们提出了包括元素召回率、元素精确率和语义相似度在内的指标，用于解析的定量评估。

- 我们还引入了用户界面上的多目标定位能力和对不存在元素的拒绝能力，从而增强真实世界场景中的鲁棒性和可靠性。

2.2.1

相关工作

通用 MLLM

近年来，通用 MLLM 取得了快速进展。GPT-4V（Yang et al. 2023）和 Gemini（Team et al. 2023）通过大规模网络级数据以及基于人类反馈的强化学习提升了性能，实现了强大的零样本视觉理解和推理能力。随后，LLaVa（Liu et al. 2023）、QwenVL（Bai et al.

2023）和 InternVL（Chen et al. 2024b）等开源模型利用精选数据集上的指令微调，高效地将 LLM 与视觉输入对齐。它们以端到端方式融合视觉编码器和语言解码器，支持包括音频和视频在内的多种模态。尽管取得了这些进展，通用 MLLM 主要是在自然图像而非 GUI 图像上训练的，因此在 GUI 特定场景中表现出有限的感知能力。

2.2

用于 GUI 感知的 MLLM

为了实现 GUI 感知，若干代表性工作（Cheng et al. 2024; Gou et al. 2024; Wu et al. 2024; Yang et al. 2024; You et al. 2024; Xu et al. 2024; Qin et al. 2025）探索了使用从用户界面中提取的大规模文本-位置对来微调 MLLM。Seeclick（Cheng et al. 2024）首次仅基于用户界面实现了自动 GUI 感知，并提出了一个多平台基准来评估 GUI 定位。Ferret-UI 系列（You et al. 2024; Li et al. 2024）使用动态分辨率策略放大界面细节，以增强视觉感知。Aguvis（Xu et al. 2024）和 UI-TARS（Qin et al. 2025）收集了大量标注数据，并利用推理范式实现了更强的 GUI 感知能力。为了充分利用用户界面中元素的语义与位置信息，OmniParser（Wan et al. 2024）利用强大的

用户界面

用户指令 请提供以下命令的位置：buy the tickets、play the music、check my account。

MLLM ☒ 视觉编码器 ☒ ☒

LoRA ☒

☒ 分词器与嵌入

大语言模型

冻结 可训练 文本 token [REJ] token [VG] token

LoRA ☒

Token 路由器 可视化 解码

仅训练 仅推理 训练与推理

☒

视觉适配器

☒

元素匹配器

坐标解码器

buy the tickets null play the music [0.75, 0.85, 0.82, 0.88]

check my account [0.85, 0.11, 0.96, 0.16]

图 2: SparkUI-Parser 的总体架构。token 路由器首先将 MLLM 的输出 token 分类为文本 token 和特殊 token ([VG] 与 [REJ])。文本 token 由 MLLM 处理。[VG] token 的特征与视觉适配器特征被联合解码以获得精确坐标, 而 [REJ] token 被丢弃。元素匹配器在训练期间将预测元素与真实标注对齐, 以避免生成元素顺序的影响。专家模型来提取图标和文本, 之后应用 GPT4V (Yang et al. 2023) 生成每个元素的功能。尽管这些方法取得了一些进展, 但仍然存在以下问题: 1) 它们只实现了对预定义元素的定位, 当要求定位一个不存在的元素时无法正确响应。2) 它们的坐标建模方式是离散的, 牺牲了定位精度和推理速度。3) 它们无法对整个界面进行解析, 或者需要使用额外工具以非端到端的方法完成解析。我们提出了一种端到端且连续的坐标建模框架, 能够从用户界面中解析所有元素, 并正确处理不存在的元素, 从而为 GUI 界面提供高精度、高效率且有效的感知。

3

方法

当前基于 MLLMs 的 GUI 感知方法通常以离散文本 token 的形式生成坐标。这种基于词表映射的离散表示在一定程度上可能导致 grounding 不准确, 从而与基于连续空间坐标回归的传统视觉 grounding 方法相比存在性能差距。为缩小这一差距并提升准确性, 我们设计了一种高效的先路由后预测 (route-then-predict) 框架, 该框架将语义理解与坐标优化解耦, 同时利用 MLLMs 原生的空间感知能力, 而不注入额外的感知模块。SparkUI-Parser 的整体架构如图 2 所示。

3.1

模型架构

该框架由一个 MLLM、一个 token 路由器、一个视觉适配器、一个坐标解码器和一个元素匹配器组成。给定一条用户指令和一个用户界面, token 路由器将 MLLM 的输出 token 分类为文本 token 和特殊 token (包括 [VG] token 和 [REJ] token)。随后, 来自 [VG] token 和视觉适配器的多模态特征被输入坐标解码器, 以生成精确坐标, 而 [REJ] token 则被直接丢弃。文本 token 默认使用 MLLM 进行解码。在训练阶段, 采用元素匹配器来建立预测元素与真实标注元素之间的语义和空间对应关系, 从而有效缓解输出顺序变化的影响。接下来, 我们对这一过程进行详细阐述。MLLM 尽管预训练 MLLM 具有良好的多模态理解能力, 但其缺乏理解和定位 GUI 界面的能力。为了增强 MLLM 的 GUI 感知能力, 我们使用 LoRA (Hu et al. 2022) 对预训练 MLLM 的视觉部分和语言部分进行微调。用户指令 Tinstruction 和界面 I 首先由微调后的 MLLM FMLLM 处理, 以获得语义特征 ftoken, 如下所示: ftoken = FMLLM (I, Tinstruction)。(1) Token 路由器 为了基于离散 token 对连续坐标建模, 我们通过 token 路由器将 MLLM 生成的输出 token 划分为语义 token 和位置 token。基于输出 token 的 logits, 我们识

别位置 token, 包括 [VG] token 和 [REJ] token。[VG] token 指代目标的边界框, 并且

输入

输入

大语言模型

大语言模型

。

7

5

Grounding  DINO

Transformer Block × N

[VG] 特征 0

检测到的图标

视觉特征

[VG]

icon1 [0.084,0.019,0.140,0.048] ... icon5
[0.740,0.714,0.919,0.806] icon6 [0.088,0.901,0.902,0.963]

检测到的文本

FFN Paddle OCR

0.75

Dashboard [0.076,0.078,0.911,0.120]

Focus mode [0.456,0.078,0.911,0.120] ... Turn on Focus mode
[0.287,0.921,0.706,0.947]

步骤 1: NMS



图 3：先前方法（左）与我们的方法（右）在坐标生成方面的比较。我们利用特征而非多个离散 token 来获得连续坐标值，从而提高 grounding 的精度并加快推理速度。请注意，为了简化可视化，这里只显示了一个值。事实上，在我们的方法中，一个 [VG] token 表示一个边界框的坐标。[REJ] token 指代不存在的位置。[REJ] token 随后被丢弃，而 [VG] token 需要进一步处理，以获得每个元素的准确坐标。语义 token 通过 MLLM 的标准自回归生成被解码为自然语言文本。视觉适配器 与其利用大规模外部视觉编码器来提供视觉特征，以准确确定 GUI 中元素的位置，我们利用 MLLM 的视觉编码器 FViE 产生的视觉特征，并微调一个视觉适配器 Fadapter，以进行更精确的 GUI 任务特定定位，进而输出增强后的视觉特征 fvision。其形式化表示为 $fvision = Fadapter(FViE(I))$ 。

(2)

这使得视觉信息能够与 MLLM 的感知进行一致且高效的整合，从而增强系统的整体性能和适应性。坐标解码器 我们引入了一个轻量级坐标解码器，以支持更精确地生成连续坐标值。我们提取 LLM 最后一层中对应于 [VG] token 的 embedding 作为文本特征。连同来自视觉适配器的视觉特征，这些文本特征被输入坐标解码器 Fdecoder，以获得每个目标元素的精确边界框坐标 OBBox。该过程可形式化表示为 $OBBox = Fdecoder(ftoken[V G], fvision)$ 。

(3)

视觉信息与文本信息的融合使模型能够同时利用来自文本的语义理解和来自视觉特征的空间感知，从而准确确定 GUI 中元素的位置。如图 3 所示，通过使用基于 transformer 的架构预测坐标，MLLM 在连续视觉 grounding 任务中基于词表预测的局限性得到缓解，从而提高了 grounding 精度。另一方面，对于一个边界框，只需生成一个特殊 token；与使用 MLLMs 直接输出边界框相比，这压缩了 token 数量，因此加快了 grounding 和 parsing 的推理速度。元素匹配器 我们观察到，文本输出固有的随机性对准确的损失

icon1 return [0.084,0.019,0.140,0.048] text1 Dashboard
[0.076,0.078,0.426,0.120] text2 Focus mode
[0.456,0.078,0.911,0.120] text3 Put your phone away

[0.167,0.617,0.823,0.656] ... icon5 add a new focus time
[0.740,0.714,0.919,0.806] text11 Tap to try (1 min)
[0.369,0.856,0.622,0.879] icon6 Turn on Focus mode
[0.088,0.901,0.902,0.963] text12 Turn on Focus mode
[0.287,0.921,0.706,0.947]

步骤 2：MLLM 语义 步骤 3：人工检查

图 4：ScreenParse 中数据标注的示例。对所有元素的元素类型、语义和边界框进行了标注。

训练期间的计算，因为预测结果与真实标注之间可能存在元素错位。为缓解这一问题，我们引入了一个元素匹配器，基于预定义的匹配分数，使用一种结合语义和位置的改进匈牙利匹配算法，高效地将每个真实标注元素与其最接近的预测对应项配对，该算法将在第 3.2 节中详细说明。该方法确保模型只因对应元素之间的差异而受到惩罚，而不会受到元素生成顺序的影响。该方法不仅简化了训练过程，还增强了模型处理多样且复杂 GUI 布局的能力，使其更能适应真实世界场景。

3.2

训练目标

为了在包含多种指令类型的复杂任务上训练该框架，而不仅仅是一个简单的定位任务，我们精心设计了一组目标，以确保训练期间稳定收敛。输出可能包含多个元素及其对应的边界框，并且输出元素的顺序与真实标注元素的顺序之间可能存在不一致。受目标检测方法 DETR (Carion et al. 2020) 的启发，我们首先使用一种改进的匈牙利匹配算法，在预测元素与真实标注之间计算最优二分匹配，该算法同时考虑语义属性和空间属性；随后，我们最小化相应的逐元素损失函数。令 $y = n \{(t_i, bio)\}$ 表示元素的真实标注集合， $y_{\boxtimes} = (t_{\boxtimes}^i, b_{\boxtimes}^i)$ 表示预测集合。 t_i 和 $t_{\boxtimes}^i$ 是第 i 个元素文本的语义特征，而 b_i 和 $b_{\boxtimes}^i$ 表示其边界框。为了为真实标注元素 y_i 找到匹配项，我们使用匹配

模型

数据

移动端

桌面端

网页端

文本

图标

文本

图标

文本

图标

平均

时间(s)

ScreenSpot GPT-4o Claude Computer Use Gemini 2.0 (Project
Mariner) Qwen2.5-VL-7B InternVL2.5-8B CogAgent-18B
SeeClick-9.6B OmniParser UGround-7B OS-Atlas-7B Aguviz-7B
SparkUI-Parser-8B

400K 1M 1.3M 2.3M 1M 515K

22.6 97.1 82.8 67.0 78.0 93.9 82.8 93.0 95.6 94.9

24.5 81.2 58.5 24.0 52.0 57.0 60.3 72.9 77.7 83.8

20.2 86.6 47.4 74.2 72.2 91.3 82.5 91.8 93.8 95.9

11.8 70.0 28.6 20.0 30.0 63.6 63.6 62.9 67.1 80.7

9.2 87.4 47.4 70.4 55.7 81.3 80.4 90.9 88.3 89.6

8.8 78.6 26.7 28.6 32.5 51.0 70.4 74.3 75.2 82.9

**18.3 83.0 84.0 84.9 48.6 47.4 53.4 73.0 73.3 82.5 84.4
88.0**

0.763 0.812 1.112 0.872 0.821 0.782 0.791 0.173

ScreenSpot-v2 GPT-4o Qwen2.5-VL-7B InternVL2.5-8B
CogAgent-18B SeeClick-9.6B UGround-7B OS-Atlas-7B
Aguvis-7B SparkUI-Parser-8B

400K 1M 1.3M 2.3M 1M 515K

26.6 99.0 84.5 69.3 78.4 84.5 95.2 95.5 96.9

24.2 84.4 58.3 27.0 50.7 61.6 75.8 81.5 87.7

24.2 87.6 48.5 75.8 70.1 85.1 90.7 93.3 95.9

19.3 65.7 30.0 20.7 29.3 61.4 63.6 77.9 77.1

12.8 90.2 43.6 74.4 55.2 84.6 90.6 91.0 93.4

11.8 79.8 27.1 31.5 32.5 71.9 77.3 77.8 86.2

20.1 86.5 48.7 52.8 55.1 76.3 84.1 87.3 89.5

0.756 0.793 1.104 0.864 0.805 0.774 0.779 0.168

Table 1: GUI grounding accuracy comparison of various methods on ScreenSpot and ScreenSpot-v2. In each column on different benchmarks, the **best** and second best performance are marked, where “-” indicates that the information cannot be obtained due to missing values or API usage.

cost:

$$\sigma = \begin{cases} \arg \min_j \mathcal{C}_{match}(y_i, \hat{y}_j) & \text{if } \mathcal{C}_{match}(y_i, \hat{y}_j) > \mu, \\ \emptyset & \text{otherwise,} \end{cases}$$

where $\mathcal{C}_{match} = \lambda_{IoU} \mathcal{L}_{IoU}(b_i, \hat{b}_j) + \lambda_{sem} \mathcal{L}_{sem}(t_i, \hat{t}_j)$. (4)

μ is the threshold to evaluate whether two elements match, which is experimentally set to 0.55.

After finding a match for each ground truth element, we use cross-entropy loss for the text, $\mathcal{L}_1$ loss and IoU loss

表 1: 各种方法在 ScreenSpot 和 ScreenSpot-v2 上的 GUI 定位准确率比较。在不同基准的每一列中，标出了最佳和次佳性能，其中“-”表示由于缺失值或 API 使用原因无法获得该信息。

4

代价: (

$\arg \min \mathcal{C}_{match}(y_i, y \boxtimes j)$

if $\mathcal{C}_{match}(y_i, y \boxtimes j) > \mu$,

j

$\sigma = \emptyset$

otherwise,

其中 $\mathcal{C}_{match} = \lambda_{IoU} \mathcal{L}_{IoU}(b_i, b \boxtimes j) + \lambda_{sem} \mathcal{L}_{sem}(t_i, t \boxtimes j)$)。 (4) μ 是用于评估两个元素是否匹配的阈值，在实验中设置为 0.55。在为每个真实标注元素找到匹配项后，我们对文本使用交叉熵损失，对边界框使用 L1 损失和 IoU 损失 (Rezatofighi et al. 2019) : $\mathcal{L}_i = \lambda_{CE} \mathcal{L}_{CE}(t_i, t \boxtimes \sigma) + \lambda_1 \mathcal{L}_1(b_i, b \boxtimes \sigma) + \lambda_{IoU} \mathcal{L}_{IoU}(b_i, b \boxtimes \sigma)$ 。 (5) 由于每个元素都包含语义和坐标优化，对于包含 N_{gt} 个元素的真实标注集合，总损失定义为: $\mathcal{L} =$

$N_{gt} \times$

$\mathcal{L}_i$

(6)

i

这些训练目标有助于从元素数量不确定的复杂任务中进行更稳定的学习。模型能够更好地应对不同用户界面中元素数量变化所带来的挑战，从而提升性能和鲁棒性。

基准

在本节中，为了构建一个解析基准 ScreenParse，我们介绍数据收

4 Benchmark

In this section, to construct a parsing benchmark, ScreenParse, we introduce the data collection and annotation method, and provide a statistical overview of our benchmark. We also propose metrics to evaluate the performance of GUI parsing.

4.1 Data Collection and Annotation

We construct the parsing benchmark across different domains and applications to ensure diversity in terms of design, functionality, and complexity. Based on the open-source 集与标注方法，并提供该基准的统计概览。我们还提出了用于评估 GUI 解析性能的指标。

4.1

数据收集与标注

我们在不同领域和应用中构建解析基准，以确保其在设计、功能和复杂性方面具有多样性。基于开源 ScreenSpot 基准 (Cheng et al. 2024)，我们重新标注了其中所有界面。每个界面都标注了元素类型、语义标签和边界框。为了覆盖更广泛的用户界面和语言，我们还以相同方式标注了若干常见中文应用的用户界面。具体而言，我们使用专家模型对用户界面进行预处理，并人工检查质量。对于图像中的所有图标元素，我们使用开放域检测模型 Grounding DINO (Liu et al. 2024)，并使用 Paddle-OCR (Authors 2020) 提取所有文本元素。对于可能重叠的边界框，我们使用非极大值抑制 (NMS) 进行过滤。仍可能有一些图标缺乏语义，因此我们使用预训练 MLLM 为其补充语义。随后，将对预处理后的数据进行人工检查，以纠正框和

模型

Func2Box

Text2Box

平均

22.1 44.3 59.8 17.2 53.6 56.8 60.8 79.1 80.2

19.9 44.0 59.3 24.2 60.7 66.7 76.5 76.5 82.9

21.0 44.2 59.6 20.7 57.2 61.8 68.7 77.8 81.6

GPT-4o GPT-4o with grounding Qwen2.5-VL-7B InternVL2.5-8B

OS-Atlas-7B UI-TARS-7B Aguis-7B AgentCPM-GUI-8B

SparkUI-Parser-8B

Table 2: GUI grounding accuracy comparison of various methods on CAGUI-Grounding. The **best** and second best performance are marked.

description information. Fig. 4 illustrates the complete data-processing and annotation pipeline

表 2：各种方法在 CAGUI-Grounding 上的 GUI 定位准确率比较。标出了最佳和次佳性能。描述信息。图 4 展示了完整的数据处理和标注流水线。

4.2

数据统计

该基准包含英文和中文用户界面，由约 800 张图像组成（每种语言 400 张）。每个界面平均标注 36 个元素。其包含了多样化的元素集合，常见类型为文本（57.5%）和图标（42.5%）。

4.3

评估指标

由于这是首次对用户界面的解析结果进行评估，我们提出从两个方面对模型进行量化：边界框性能和语义性能。边界框性能引入了两个指标来评估边界框性能：元素召回率和元素精确率，它们为模型在 GUI 解析任务中的性能提供了全面视角。元素召回率定义为正确定位的元素数量与真实标注元素总数之比，而元素精确率定义为正确定位的元素数量与预测元素总数之比。较高的元素召回率表明模型能够有效识别元素的位置，确保很少遗漏实际元素。较高的精确率表明模型定位出的多数元素确实存在于用户界面中，从而最小化错误定位的数量。语义性能引入语义相似度来评估语义性能，其通过计算模型预测语义与匹配边界框的真实标注之间的一致性来实现，涵盖文本相似度和图标描述相似度。语义相似度评估模型为已定位元素提供语义准确描述的能力，这对于需要深度 GUI 理解的任务至关重要，例如智能用户交互和上下文感知辅助。

5.5.1

实验

实验设置

网络架构 在我们的工作中，我们使用 InternVL2.5-8B（Chen et al. 2024a）作为基础 MLLM FMLLM，并且视觉适配器 Fadapter 是一个通道数为 [4096, 2048, 1024, 256] 的 MLP 模块。坐标解码器是一个基于 transformer 的架构，采用交叉注意力机制，其中隐藏状态维度为 256，并最终通过解码头回归到坐标。实现细节 我们采用 8 块 NVIDIA 80G A100 来训练整个框架，并且在训练期间解冻视觉编码器。LoRA（Hu et al. 2022）被用于微调 MLLM 中的视觉和语言部分，其中 lora-rank r 和 lora-alpha α 分别设置为 16 和 32。我们还采用 CosineLRScheduler 作为学习率调度器，其中学习率和 warmup 比例分别设置为 $3e-5$ 和 0.03。在训练过程中，式 5 中

CosineLRScheduler as the learning rate scheduler, where the learning rate and warmup ratio are set to $3e^{-5}$ and 0.03 respectively. In the training process, λ_{CE} , λ_1 and λ_{IoU} in Eq. 5 are set to 2.0, 4.0 and 1.0 respectively, while λ_{IoU} and λ_{sem} in element matcher are both set to 1.0 in Eq. 4.

Datasets Our training data is composed of open-source 的 λ_{CE} 、 λ_1 和 λ_{IoU} 分别设置为 2.0、4.0 和 1.0，而元素匹配器中式 4 的 λ_{IoU} 和 λ_{sem} 均设置为 1.0。数据集 我们的训练数据由开源英文数据集和自标注中文数据集组成，包括移动端、桌面端和 Web 领域的 515k 个训练样本。对于英文数据集，我们为定位任务采样 400k 个训练样本，并额外标注 5k 个解析样本。对于中文数据集，我们在自收集的用户界面上标注了 100k 个定位样本和 10k 个解析样本。（更多数据集细节见补充材料。）此外，为了优化训练效率并增强模型鲁棒性，我们实现了一种多目标指令范式，其中每个训练实例随机包含多个目标，不同于传统的单目标方法。基线 我们将 SparkUI-Parser 与各种基线进行比较，包括闭源商业模型 GPT4o（Hurst et al. 2024）、Claude Computer Use（Anthropic 2024）、Gemini 2.0（Project Mariner）（GoogleDeepmind 2024），以及开源基础模型 Qwen2.5-VL 系列（Bai et al. 2025）、InternVL2.5 系列（Chen et al. 2024a）和学术 GUI 模型 SeeClick、CogAgent（Hong et al. 2024）、Aguvis（Xu et al. 2024）、OS-Atlas（Wu et al. 2024）、UGround（Gou et al. 2024）和 UI-TARS（Qin et al. 2025）。

5.2

定位结果

为了评估模型的 GUI 定位能力，我们使用了以下基准：1. ScreenSpot，一个 GUI 单步定位基准；2. ScreenSpot-v2（Wu et al. 2024），一个基于 ScreenSpot 的重新标注版本，修正了一些坐标和指令错误；3. CAGUI-Grounding（Zhang et al. 2025），一个中文移动端文本和功能定位评估基准。与采用文本自回归进行视觉定位的传统模型相比，SparkUI-Parser 利用基于 transformer 的坐标解码器来聚合视觉和文本特征，并将它们直接投影为连续坐标表示。如表 1 所示，我们的方法仅使用少量数据便取得了 SOTA 结果。在 token router 和坐标解码器的帮助下，我们的推理速度达到最快，这表明其在多样化应用场景的实际部署中具有更大潜力。同时，SparkUI-Parser 在表 2 的 CAGUI-grounding 上优于其他模型。评估

英文

模型 GPT-4o Claude Computer Use Qwen2.5-VL-7B
Qwen2.5-VL-72B InternVL2.5-8B InternVL2.5-78B
SparkUI-Parser-8B

中文

时间(s)

召回率

精确率

SemanticSim

召回率

精确率

SemanticSim

5.7 17.1 18.8 22.9 11.7 20.4 77.2

11.7 35.3 36.6 43.7 30.4 38.6 77.9

0.586 0.758 0.596 0.685 0.615 0.628 0.918

4.9 19.3 25.0 42.4 28.6 40.8 87.1

7.8 31.0 52.6 70.5 47.9 66.4 89.5

0.406 0.511 0.854 0.867 0.812 0.836 0.946

0.534 2.748 0.558 2.822 0.154

Table 3: GUI parsing evaluation of various methods on ScreenParse. The **best** and second best performance are marked, where “-” indicates the time consumption cannot be obtained because of API usage.

Model	ScreenSpot-v2		ScreenParse		
	Text Avg.	Icon Avg.	Recall Avg.	Precision Avg.	SemanticSim Avg.
full model	95.4	83.7	82.2	83.7	0.932
w/o coordiante decoder	91.2	80.2	77.6	78.2	0.930
w/o vision adapter	22.1	17.8	5.4	4.6	0.915
w/o element matcher	94.7	83.1	79.5	80.4	0.814
w/o parsing dataset	95.0	83.2	20.2	39.2	0.751

表 3：各种方法在 ScreenParse 上的 GUI 解析评估。标出了最佳和次佳性能，其中 “-” 表示由于 API 使用而无法获得时间消耗。模型完整模型 w/o coordiante decoder w/o vision adapter w/o element matcher w/o parsing dataset

ScreenSpot-v2

ScreenParse

文本平均

图标平均

召回率平均

精确率平均

SemanticSim 平均

95.4 91.2 22.1 94.7 95.0

83.7 80.2 17.8 83.1 83.2

82.2 77.6 5.4 79.5 20.2

83.7 78.2 4.6 80.4 39.2

0.932 0.930 0.915 0.814 0.751

Table 4: Ablation study to analyze the effectiveness of each component in our method, for conciseness and clarity of presentation, we primarily focus on results on ScreenSpot-v2 and ScreenParse.

ation on multiple GUI grounding benchmarks demonstrates that our method effectively supports and adapts to multilingual user interfaces and various instruction tasks.

5.3 Parsing Results

表 4: 消融研究, 用于分析我们方法中每个组件的有效性; 为使呈现简洁清晰, 我们主要关注 ScreenSpot-v2 和 ScreenParse 上的结果。在多个 GUI 定位基准上的评估表明, 我们的方法能够有效支持并适应多语言用户界面和各种指令任务。

将坐标形式转化为连续空间特征建模, 促进 MLLM 更好地完成坐标相关任务。

5.3

视觉适配器的影响 表 4 的第三行显示, 当移除视觉适配器、仅使用文本特征并通过自注意力来回归坐标时, 准确率大幅下降, 这表明用户界面元素的定位需要充分的视觉特征交互。

解析结果

为了评估模型对用户界面元素语义和坐标的全局感知能力, 我们使用所提出的基准 ScreenParse, 该基准包含英文和中文用户界面。预测元素的平均耗时是在 NVIDIA 80G A100 上通过 transformer 框架获得的。如表 3 所示, SparkUI-Parser 在解析各种用户界面方面表现出优越性能。商业模型以及大规模模型只能定位少量元素, 显示出较差的召回率和精确率。同时, 它们所定位元素的语义相似度较低, 表现出对元素语义理解的偏差。通过解耦解析任务中的语义和坐标, 并在训练过程中使用元素匹配器来保持正确的元素优化, 我们的模型相较其他模型在 ScreenParse 上取得了显著提升, 同时也显著降低了单个元素的平均推理时间。

5.4

消融研究

我们进行消融研究, 以分析并验证我们方法中每个组件的有效性。坐标解码器的影响 表 4 的第二行表明, 与 MLLM 的自回归生成方法相比, 我们提出的解耦框架在各种基准上均有不同程度的提升。这也验证了将离散词表

元素匹配器的影响 表 4 的第四行表明, 移除元素匹配器会导致解析准确率下降, 这主要是由于 MLLM 生成的随机性以及目标不匹配造成的。这说明元素匹配器在将模型输出与真实值对齐方面发挥着关键作用, 减少了对学习完全一致输出顺序的关注。解析数据集的影响 引入带标注的解析数据集显著增强了模型的全局结构感知能力。如表 4 的第五行所示, 引入解析数据集在结构感知和定位精度两方面都带来了显著提升, 同时也略微提升了 GUI 定位性能。

form of coordinates into continuous spatial feature modeling promotes MLLM to better complete coordinate-related tasks.

Impact of Vision Adapter The third row of Tab. 4 shows that when the visual adapter is removed and only text fea-

6

结论

本文提出了一种“先路由后预测”的框架, 用于解耦用户界面元素的语义感知与位置感知。通过利用专门的坐标解码器将位置特征映射到连续坐标空间, 而不是通过传统文本自回归方法生成坐标 token, 我们的方法在以下基准上取得了

最先进的性能, 同时具有最高的推理效率

: ScreenSpot、ScreenSpot-v2、CAGUI-Grounding 和 ScreenParse。与此同时, 我们在 GUI 场景中引入了拒绝机制, 以避免模型生成错误的幻觉结果, 并增强鲁棒性和可靠性。此外, 我们发布了一个用户界面解析基准, 用于评估模型对用户界面全局语义和位置的感知能力, 以促进社区的进一步研究。

参考文献 Anthropic. 2024. Developing a computer use model. Technical report, Anthropic. Authors, P. 2020. PaddleOCR, Awesome multilingual OCR toolkits based on PaddlePaddle. <https://github.com/PaddlePaddle/PaddleOCR>. Bai, J.; Bai, S.; Yang, S.; Wang, S.; Tan, S.; Wang, P.; Lin, J.; Zhou, C.; and Zhou, J. 2023. Qwen-VL: A Versatile VisionLanguage Model for Understanding, Localization, Text Reading, and Beyond. arXiv preprint arXiv:2308.12966. Bai, S.; Chen, K.; Liu, X.; Wang, J.; Ge, W.; Song, S.; Dang, K.; Wang, P.; Wang, S.; Tang, J.; et al. 2025. Qwen2. 5-vl technical report. arXiv preprint arXiv:2502.13923. Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; and Zagoruyko, S. 2020. End-to-end object detection with transformers. In European conference on computer vision, 213–229. Springer. Chen, Z.; Wang, W.; Cao, Y.; Liu, Y.; Gao, Z.; Cui, E.; Zhu, J.; Ye, S.; Tian, H.; Liu, Z.; et al. 2024a. Expanding Performance Boundaries of Open-Source Multimodal Models with Model, Data, and Test-Time Scaling. arXiv preprint arXiv:2412.05271. Chen, Z.; Wu, J.; Wang, W.; Su, W.; Chen, G.; Xing, S.; Zhong, M.; Zhang, Q.; Zhu, X.; Lu, L.; et al. 2024b. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 24185–24198. Cheng, K.; Sun, Q.; Chu, Y.; Xu, F.; Li, Y.; Zhang, J.; and Wu, Z. 2024. SeeClick:

Harnessing gui grounding for advanced visual gui agents. arXiv preprint arXiv:2401.10935. GoogleDeepmind. 2024.

Gemini-2.0 (project mariner). Technical report, GoogleDeepmind. Gou, B.; Wang, R.; Zheng, B.; Xie, Y.; Chang, C.; Shu, Y.; Sun, H.; and Su, Y. 2024. Navigating the digital world as humans do: Universal visual grounding for gui agents. arXiv preprint arXiv:2410.05243. Hong, W.; Wang, W.; Lv, Q.; Xu, J.; Yu, W.; Ji, J.; Wang, Y.; Wang, Z.; Dong, Y.; Ding, M.; et al. 2024. Cogagent: A visual language model for gui agents. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 14281–14290. Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W.; et al. 2022. Lora: Low-rank adaptation of large language models. ICLR, 1(2): 3.

Hurst, A.; Lerer, A.; Goucher, A. P.; Perelman, A.; Ramesh, A.; Clark, A.; Ostrow, A.; Welihinda, A.; Hayes, A.; Radford, A.; et al. 2024. Gpt-4o system card. arXiv preprint arXiv:2410.21276. Li, Z.; You, K.; Zhang, H.; Feng, D.; Agrawal, H.; Li, X.; Moorthy, M. P. S.; Nichols, J.; Yang, Y.; and Gan, Z. 2024. Ferret-ui 2: Mastering universal user interface understanding across platforms. arXiv preprint arXiv:2410.18967. Liu, H.; Li, C.; Wu, Q.; and Lee, Y. J. 2023. Visual Instruction Tuning. Liu, S.; Zeng, Z.; Ren, T.; Li, F.; Zhang, H.; Yang, J.; Jiang, Q.; Li, C.; Yang, J.; Su, H.; et al. 2024. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In European Conference on Computer Vision, 38–55. Springer. Qin, Y.; Ye, Y.; Fang, J.; Wang, H.; Liang, S.; Tian, S.; Zhang, J.; Li, J.; Li, Y.; Huang, S.; et al. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. arXiv preprint arXiv:2501.12326. RezaTofighi, H.; Tsoi, N.; Gwak, J.; Sadeghian, A.; Reid, I.; and Savarese, S. 2019. Generalized intersection over union: A metric and a loss for bounding box regression. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 658–666.

Team, G.; Anil, R.; Borgeaud, S.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; Millican, K.; et al. 2023. Gemini: a family of highly capable multimodal models. arXiv preprint arXiv:2312.11805. Wan, J.; Song, S.; Yu, W.; Liu, Y.; Cheng, W.; Huang, F.; Bai, X.; Yao, C.; and Yang, Z. 2024. Omniparser: A unified framework for text spotting key information extraction and table recognition. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 15641–15653. Wu, Z.; Wu, Z.; Xu, F.; Wang, Y.; Sun, Q.; Jia, C.; Cheng, K.; Ding, Z.; Chen, L.; Liang, P. P.; et al. 2024. Os-atlas: A foundation action model for generalist gui agents. arXiv preprint arXiv:2410.23218.

Xu, Y.; Wang, Z.; Wang, J.; Lu, D.; Xie, T.; Saha, A.; Sahoo, D.; Yu, T.; and Xiong, C. 2024. Aguis: Unified pure vision agents for autonomous gui interaction. arXiv preprint arXiv:2412.04454. Yang, Y.; Wang, Y.; Li, D.; Luo, Z.; Chen, B.; Huang, C.; and Li, J. 2024. Aria-UI: Visual Grounding for GUI Instructions. arXiv preprint arXiv:2412.16256. Yang, Z.; Li, L.; Lin, K.; Wang, J.; Lin, C.-C.; Liu, Z.; and Wang, L. 2023. The dawn of llms: Preliminary explorations with gpt-4v (ision). arXiv preprint arXiv:2309.17421, 9(1): 1. You, K.; Zhang, H.; Schoop, E.; Weers, F.; Swearngin, A.; Nichols, J.; Yang, Y.; and Gan, Z. 2024. Ferret-ui: Grounded mobile ui understanding with multimodal llms. In European Conference on Computer Vision, 240–255. Springer. Zhang, Z.; Lu, Y.; Fu, Y.; Huo, Y.; Yang, S.; Wu, Y.; Si, H.; Cong, X.; Chen, H.; Lin, Y.; et al. 2025. AgentCPM-GUI: Building Mobile-Use Agents with Reinforcement FineTuning. arXiv preprint arXiv:2506.01391.